Recap

Earlier,

We learnt about Hard Margin SVM

↓

Hard Margin SVM didn't give any flexibility with errors.



So,

We introduced Soft Margin SVM

↓

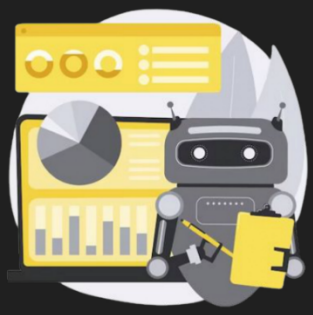
Which allows misclassifications up to a certain extent.

Optimization Function

$$min_{w,b} \frac{||w||}{2} + \frac{C}{N}\sum_{i=1}^{N}\zeta_i$$

$$s.t. (w^T x_i + b)y_i \geq 1 - \zeta_i \;\; \forall\, i : 1 \rightarrow N$$

$$\zeta_i \geq 0$$



This is known as the **Primal Form**

On solving the primal form equation we get : ***optimal w* , b****

Optimization Theory

In Optimization theory,

we have **Primal Dual Equivalence,** which states that :

For a primal form optimization problem,
There exists a dual form optimization problem.

Solving the dual form you'll arrive at the same optimal solution as the primal form.
However, the proof is challenging and math intensive.

Dual formulation of SVM

$$max_{\alpha_i} \sum_{i=1}^{N} \alpha_i - \frac{1}{2} \sum_{i=1}^{N} \sum_{j=1}^{N} \alpha_i \alpha_j y_i y_j x_i^T . x_j$$

$$\left.\begin{aligned} & s.t.\ 0 \leq \alpha_i \leq C \\ & \sum_{i=1}^{N} \alpha_i y_i = 0 \end{aligned}\right\} \quad \forall\, i : 1 \rightarrow N$$

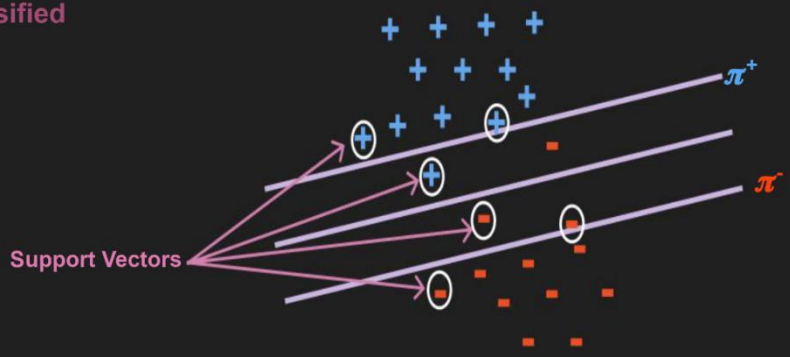
Notice that here…

- We're finding optimal α's instead of w's & b's.
- We're maximizing the equation instead of minimizing.
- x_i appears in pair i.e., $x_i^T x_j$ (not independently).

As we know,

Support Vectors are points :

- which are either within margin
- or lie on +ve / -ve hyperplane
- or are misclassified



How do we predict the data points using dual form?

Prediction equation for dual form :-

$$\underbrace{\hat{y}}_{f(x_q)} = \sum_{i=1}^{N} \alpha_i y_i \, x_i^T . x_q$$

Learnt

Query point

Query point

Training data

Class label (-1 or +1)

Say, we have 10k data points.

How many support vectors will we have?

Probably around 1-2 % i.e. 100-200 points will act as support vectors.

So, we will have :

- 9900 points with $\alpha_i = 0$
- 100 points with $\alpha_i > 0$



In order to make the prediction of query point (xq),

We just need to calculate using 100 points i.e. ***Support vectors***

- as rest of the data points have $\alpha_i = 0$
- so they won't contribute to the summation

EXAMPLE

Optimizing the dual problem and finding the α_i's

Dual Problem → Solve → Find [α_i's] > 0

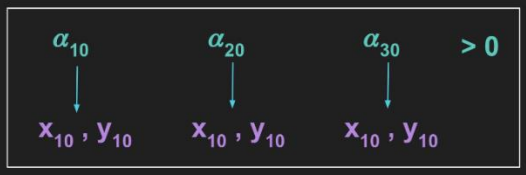
for Support Vectors

To make prediction we store,

- α_i's values and
- their corresponding x_i , y_i values

EXAMPLE

Suppose we only have 3 non-zero α values, so we store :



To predict the value of x_q :-

$$\hat{y_q} = \sum_{i=1}^{N} \alpha_i y_i \; x_i^T \cdot x_q$$

$$= \alpha_{10}\, y_{10}\, x_{10}^T x_q \quad + \quad \alpha_{12}\, y_{12}\, x_{12}^T x_q \quad + \quad \alpha_{30}\, y_{30}\, x_{30}^T x_q$$

Summarizing

At runtime we just need to store :

- α_i of Support Vectors only and
- their corresponding x_i , y_i values

Using these, we can simply make prediction for the query point x_q

So far,

- We know that SVM can't handle non linear data classification.

Let's see how to handle non linear data using SVM.....

Kernel SVM

Remember,

In dual form of SVM we saw $x_i^T x_j$, this represents similarity between two vectors.

Similarity (x_i , x_j) → Cosine Sim (x_i , x_j)

Similarity (x_i , x_j) → Kernel K(x_i , x_j)

Instead of calculating similarity using **dot product**, we will use **kernel function**.

$$max_{\alpha_i} \sum_{i=1}^{N} \alpha_i - \frac{1}{2} \sum_{i=1}^{N} \sum_{j=1}^{N} \alpha_i \alpha_j y_i y_j x_i^T . x_j$$

$$s.t.\ 0 \leq \alpha_i \leq C$$

$$\sum_{i=1}^{N} \alpha_i y_i = 0$$

$$\forall\, i : 1 \rightarrow N$$

Importance of Kernel Function

Say,

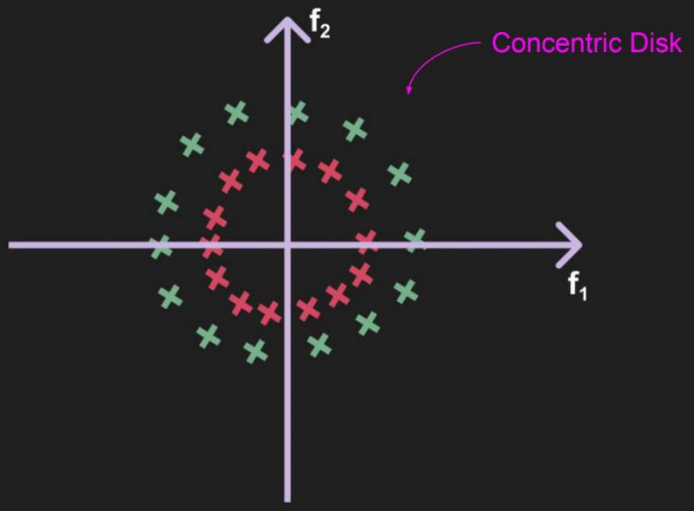
- We have Amazon's customer data but the vector representation x_i isn't available.
- All you have is how similar a customer is to another customer.



No, since logloss needs x_i

But we can use SVM in dual form as long as similarity data is available.

How this help to classify non- linear data?



Can we use logistic regression here?

Simple logistic regression will

- Always give a linear line and hence, fail to classify.
- However, by using feature engineering i.e.
- Higher degree polynomial f_1^2, f_2^2 we'll be able to classify it.

However in SVM, we have ***Kernelization***.

Kernelization transforms data from **d** to **d'** dimensions where **d' >> d**

Taking data to higher dimension makes it **linearly separable**.

Hence, making it easy to classify.

Polynomial Kernel

Polynomial Kernel is written as :

$$K(x_1, x_2) = (x_1^T x_2 + c)^m$$ → m = degree

When m = 2,

- it becomes *Quadratic Kernel*

When m = 3,

- it becomes *Cubic Kernel*

Quadratic Kernel

Assume C = 1 (for simplicity)

$$K_q(x_1, x_2) = 1 + (x_1^T x_2)^2$$

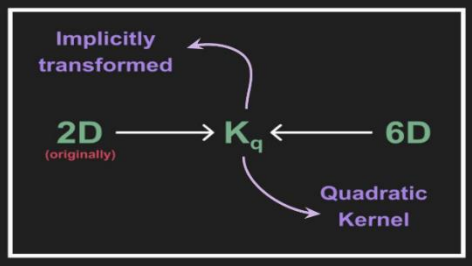
Say we have two data points x_1 , x_2 in a 2D setup

s.t. $x_1 = [x_{11}, x_{12}]$ $x_2 = [x_{21}, x_{22}]$

If we expand the equation : $K_q(x_1, x_2) = 1 + (x_1^T x_2)^2$

$$= (1 + [x_{11}\ x_{12}] \begin{bmatrix} x_{21} \\ x_{22} \end{bmatrix})^2$$

What does it signify?



Kernel trick implicitly projects data to higher dimension where it'll become separable so we don't need to explicitly add features.



Solving kernel function is equivalent to finding hyperplane in d' dimensions.

What degree of kernel to use?

Here, **degree (m)** is a hyperparameter.

If not sure about what degree to use,

- we use an general purpose kernel instead i.e. ***RBF kernel***

RBF ⟶ Radial Basis Function Kernel

$$K_{RBF}(x_1, x_2) = e^{\frac{-\|x_1 - x_2\|^2}{2\sigma^2}}$$

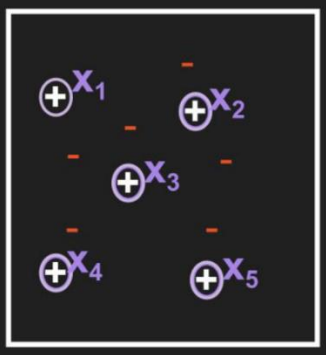
Euclidean Distance

Hyperparameter

How SVM works with RBF Kernel?

Replace $x_i^T x_j$ with $K_{RBF}(x_i, x_j)$

Now suppose, we have some data points such that:

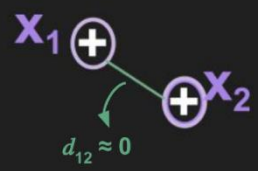


These points cannot be classified using a linear classifier.

How RBF Kernel will classify them?

Consider two points x_1 and x_2

- Since both the points are close to each other, the distance $d_{12} \approx 0$,
- Thus the kernel value will be high for them.



$K_{RBF}(x_1, x_2)$ = high

To summarize :-

We have some +ve and -ve data points in d dimension.

For **small value of** σ, in d' dimension

- RBF kernel will be grouping data points which are close to each other.

This is similar to what we learnt in KNN

Recall that :-

- for a small value of k in KNN model
- will also be making the same grouping of data.

Will outliers affect SVM?

If using SVM without any kernels hyperplane will only depend on support vectors.

∴ So, outliers will not impact the hyperplane.

However, since kernel SVMs are based on distance/ proximity and similarity.

∴ The outliers will affect them.



Why is SVM not used much?

One of the biggest problem with SVM is **its train time complexity is very high.**

$$O(n^2),\ n = \text{number of data points}$$

Also, SVMs runtime complexity depends on number of support vectors which we can't control.

